QR Code Based Attendance System

Final Capstone Project — Part I: Proposal and Prototyping

Submission Details	
Submitted by	Md Jahidul Islam
Student ID	IT24018
Course	Software Engineering and Project Management
Department	Information and Communication Technology
University	Mawlana Bhashani Science and Technology University
Supervised by	Dr. Ziaur Rahman Professor Dept. of ICT, MBSTU
Date	October 26, 2026

Repository:
https://github.com/mohammadjahidulislam/smart-attendance-proposal

Abstract

Manual and paper-based attendance in universities is slow, error-prone and hard to report on. This proposal describes the QR Code Based Attendance System, a role-based system in which teachers open class sessions, students check in, leave requests are submitted and approved online, and administrators receive course-wise attendance reports. The system automatically warns students whose attendance falls below a configurable threshold (default 75%). The proposal covers the problem, scope, requirements, risks, technology stack, budget, a Scrum-based schedule with a Gantt chart, six UML diagrams, and a prototype built from five classes of the class diagram.

Contents

1 Introduction **2**
- 1.1 Background . . . 2
- 1.2 Problem Statement . . . 2
- 1.3 Objectives . . . 2
- 1.4 Scope . . . 2
- 1.5 Unique Features . . . 2
- 1.6 Stakeholders . . . 2

2 Requirements **3**
- 2.1 Functional Requirements . . . 3
- 2.2 Non-Functional Requirements . . . 3

3 Feasibility and Risk Analysis **3**
- 3.1 Feasibility . . . 3
- 3.2 Risk Register . . . 3

4 Technology Stack, Resources and Budget **4**
- 4.1 Estimated Budget . . . 4

5 Software Process Model and Schedule **4**
- 5.1 Chosen Model: Agile (Scrum) . . . 4
- 5.2 Sprint Plan . . . 4
- 5.3 Milestones and Deliverables . . . 4
- 5.4 Gantt Chart . . . 5

6 System Design (UML Diagrams) **5**
- 6.1 Data Flow Diagram . . . 5
- 6.2 Use Case Diagram . . . 7
- 6.3 Activity Diagram . . . 8
- 6.4 Entity Relationship Diagram . . . 9
- 6.5 Class Diagram . . . 10
- 6.6 Object Diagram . . . 10

7 Prototype **11**

8 Testing and Quality Plan **12**

9 Conclusion **12**

1 Introduction

1.1 Background

Universities must track student attendance for every course, because many institutions require a minimum percentage for exam eligibility. Teachers still often use paper registers or spreadsheets, which makes proxy attendance, calculation mistakes and delayed reporting common.

1.2 Problem Statement

- Taking attendance by roll call wastes class time.
- Calculating per-student attendance percentages by hand is slow and error-prone.
- Students learn about low attendance too late to act.
- Leave and medical excuses are handled on paper and are hard to verify.
- Administrators cannot easily get department or course-wise reports.

1.3 Objectives

1. Digitise attendance recording per class session.
2. Compute attendance percentage per student per course automatically.
3. Provide online leave application and approval.
4. Send early warnings when attendance drops below the threshold.
5. Provide exportable reports (CSV/PDF) for teachers and administrators.

1.4 Scope

In scope: user management, course and session management, attendance marking, leave management, low-attendance warnings, reports, admin dashboard.
Out of scope: biometric/face recognition, payroll, examination management, native mobile apps (the web interface is responsive).

1.5 Unique Features

This project differs from a basic attendance recorder by focusing on (1) leave management with approval workflow, (2) automatic low-attendance warnings and (3) an administrator reporting dashboard.

1.6 Stakeholders

Stakeholder	Interest
Student	Check in, view own attendance, apply for leave, receive warnings
Teacher	Open sessions, mark/correct attendance, approve leave, view course reports
Administrator	Manage users and courses, view dashboards and reports
Department head	Monitor attendance trends
Developer (Md Jahidul Islam)	Design, implement, test and document the system

2 Requirements

2.1 Functional Requirements

ID	Requirement
FR1	Users shall log in with role-based access (Student, Teacher, Admin).
FR2	Admin shall create and manage users, courses and enrolments.
FR3	Teacher shall open and close a class session for a course.
FR4	Student shall mark attendance while a session is open; teacher may correct records.
FR5	System shall store status as Present, Absent, Late or Excused.
FR6	Student shall submit a leave request with reason and dates.
FR7	Teacher shall approve or reject leave; approved leave marks sessions as Excused.
FR8	System shall compute attendance percentage per student per course.
FR9	System shall notify students whose attendance is below the threshold.
FR10	System shall export course and department reports as CSV and PDF.

2.2 Non-Functional Requirements

Attribute	Requirement
Performance	Attendance marking shall complete within 2 seconds for up to 200 concurrent users.
Security	Passwords hashed; role-based authorization; HTTPS in deployment.
Usability	A new user shall mark attendance within 3 clicks after login.
Reliability	Availability of at least 99% during working hours; daily database backup.
Maintainability	Layered architecture; unit-test coverage of at least 60% for core classes.
Portability	Runs on modern browsers and on Windows/Linux servers.

3 Feasibility and Risk Analysis

3.1 Feasibility

Technical: uses mature open-source tools. **Economic:** low cost, no paid licences. **Operational:** workflow mirrors the existing manual process. **Schedule:** 12 weeks, one developer.

3.2 Risk Register

ID	Risk	Prob.	Impact	Mitigation
R1	Requirements change late	Med	High	Sprint reviews, prioritised backlog
R2	Single developer unavailable (illness)	Low	High	Buffer in each sprint, daily commits
R3	Proxy attendance by students	High	Med	Time-limited sessions, teacher correction, audit log
R4	Data loss	Low	High	Daily backups, version control
R5	Underestimated effort	Med	Med	Re-estimate each sprint, cut low-priority features
R6	Security breach / weak passwords	Low	High	Hashing, input validation, role checks

4 Technology Stack, Resources and Budget

Layer	Choice
Language	Java 17
Backend / Web	Javalin (REST API)
Database	SQLite for prototype; MySQL/PostgreSQL for deployment
Frontend	HTML, CSS, JavaScript (responsive); JavaFX for the desktop prototype
Build	Maven
Testing	JUnit 5
Version control	Git and GitHub
Documentation	LaTeX, PlantUML/TikZ

4.1 Estimated Budget

Item	Cost (BDT)
Developer effort (12 weeks, academic project)	0
Development laptop (existing)	0
Cloud hosting / domain (optional, 3 months)	3,000
Internet and miscellaneous	2,000
Contingency (10%)	500
Total	**5,500**

5 Software Process Model and Schedule

5.1 Chosen Model: Agile (Scrum)

Scrum is used because requirements (reports, notifications) are likely to evolve and the product can be delivered incrementally. Work is organised into six two-week sprints. Each sprint ends with a working increment, a review and a retrospective. The product backlog is kept in GitHub Projects.

5.2 Sprint Plan

Sprint	Dates (2026–27)	Goal and deliverable
1	2 Nov – 15 Nov	Requirements, backlog, proposal document
2	16 Nov – 29 Nov	UML design, database schema, UI wireframes
3	30 Nov – 13 Dec	Login, users, courses, sessions, attendance marking
4	14 Dec – 27 Dec	Leave management and approval workflow
5	28 Dec – 10 Jan	Percentage calculation, warnings, reports (CSV/PDF)
6	11 Jan – 24 Jan	Testing, bug fixing, deployment, final documentation

5.3 Milestones and Deliverables

ID	Week	Milestone / Deliverable
M1	2	Approved proposal and product backlog
M2	4	UML diagrams and database design complete
M3	6	Attendance module working (demo)
M4	8	Leave management working
M5	10	Reports and warnings working
M6	12	Tested system, user manual, final report

5.4 Gantt Chart

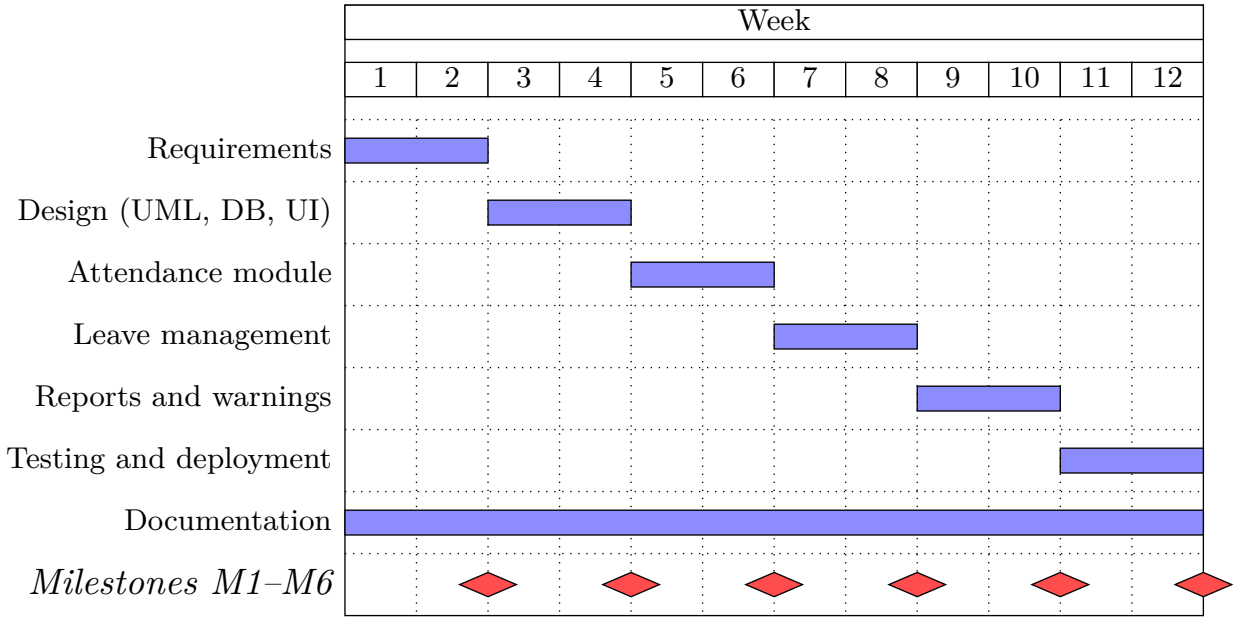


Figure 1: Project Gantt chart (12 weeks, starting 2 November 2026).

6 System Design (UML Diagrams)

6.1 Data Flow Diagram

Level 0 (context diagram).

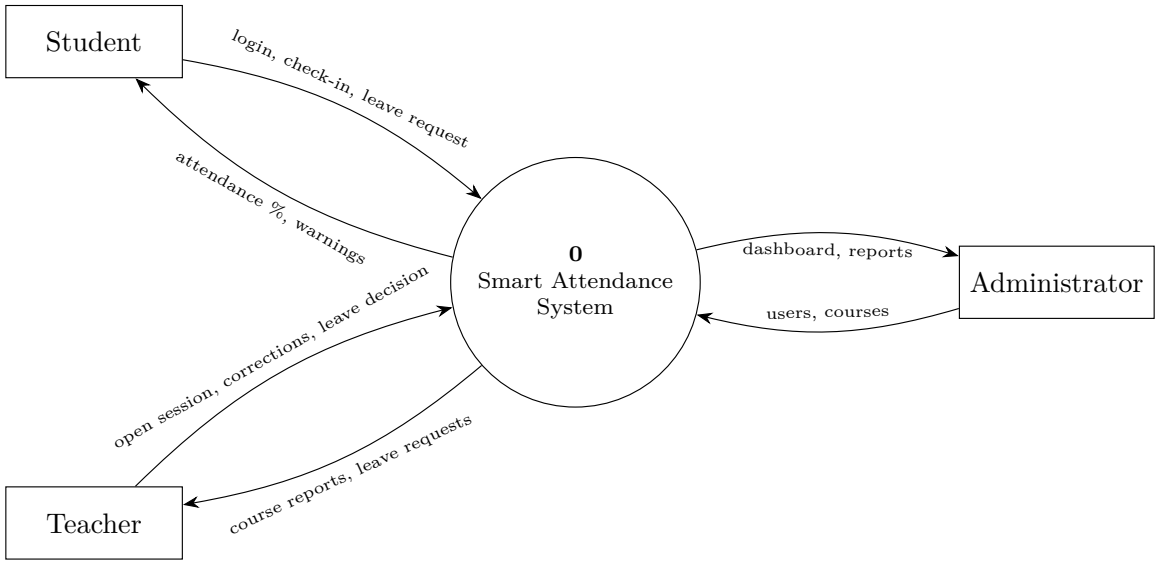


Figure 2: DFD Level 0.

Level 1.

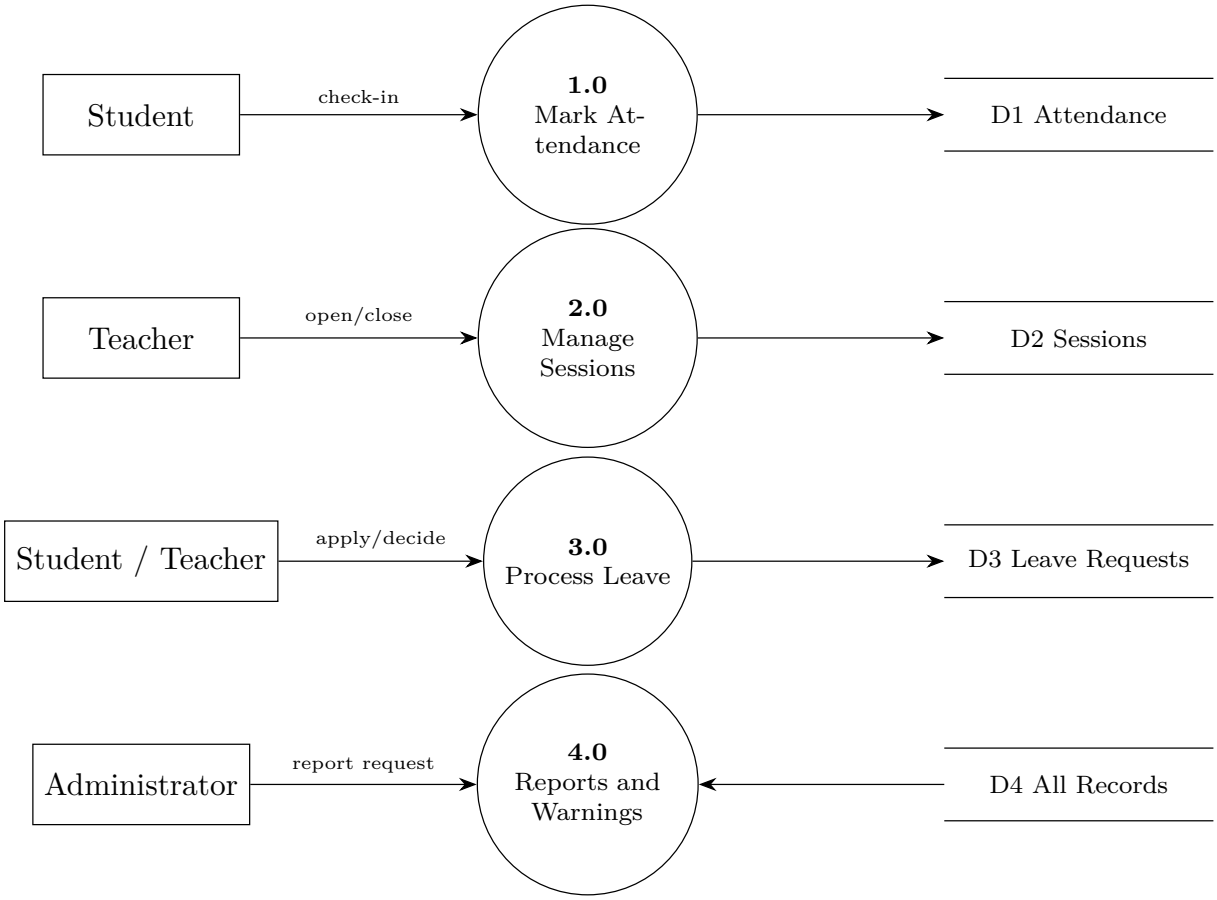


Figure 3: DFD Level 1.

6.2 Use Case Diagram

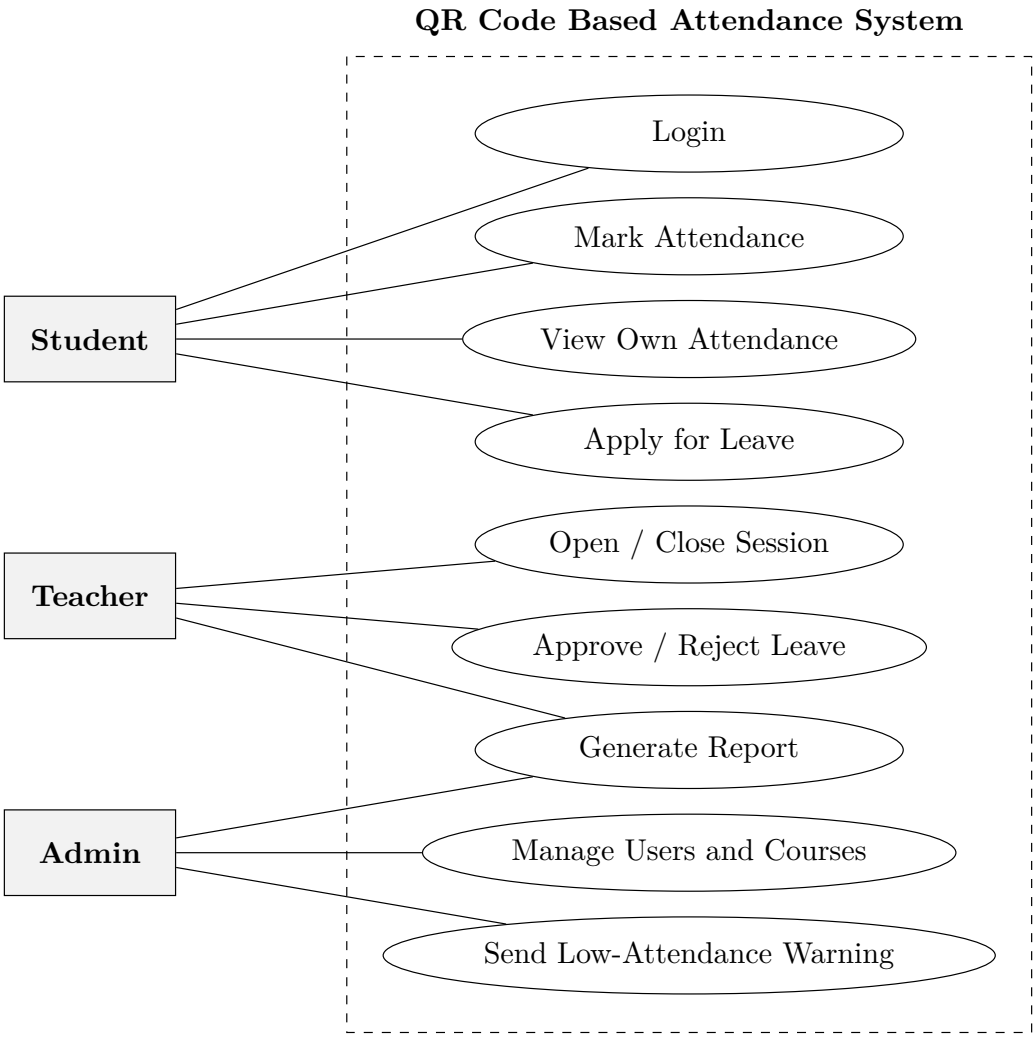


Figure 4: Use case diagram (all actors log in first).

6.3 Activity Diagram

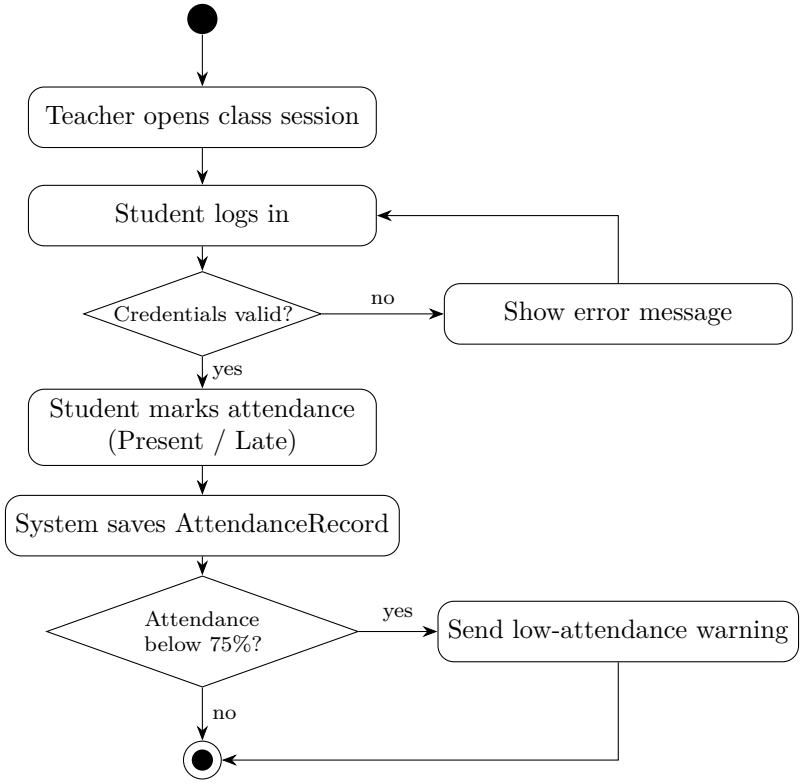


Figure 5: Activity diagram: marking attendance and low-attendance warning.

6.4 Entity Relationship Diagram

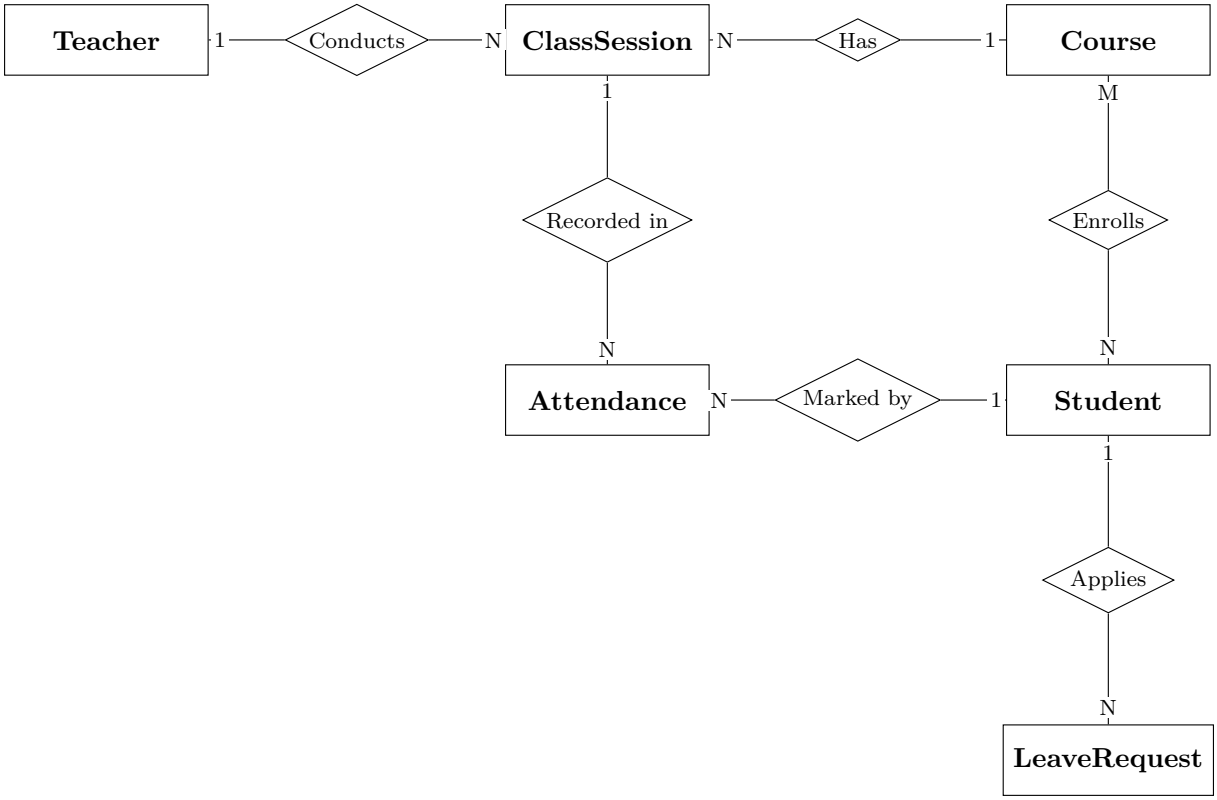


Figure 6: ER diagram (cardinalities shown on the lines).

Attributes.

Entity	Attributes (primary key in italics)
Student	*studentId*, name, department, email, passwordHash
Teacher	*teacherId*, name, email, passwordHash
Course	*courseCode*, title, credit, teacherId (FK)
ClassSession	*sessionId*, courseCode (FK), teacherId (FK), date, startTime, room, isOpen
Attendance	*attendanceId*, studentId (FK), sessionId (FK), status, markedAt
LeaveRequest	*leaveId*, studentId (FK), fromDate, toDate, reason, status, approvedBy

6.5 Class Diagram

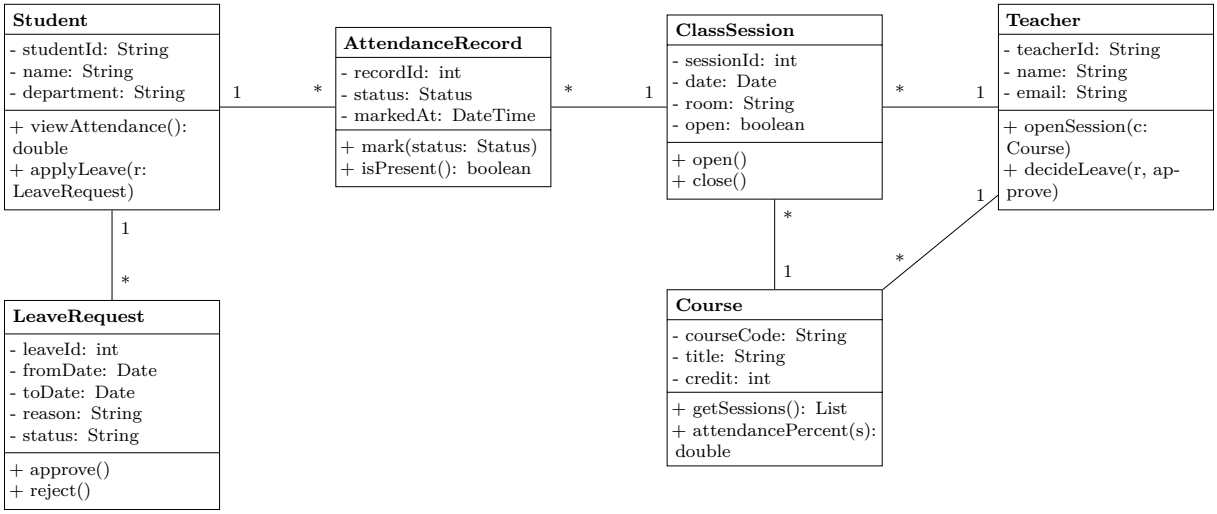


Figure 7: Class diagram. Associations: Student–AttendanceRecord, ClassSession–AttendanceRecord, Teacher–ClassSession, Course–ClassSession, Student–LeaveRequest, Teacher–Course.

6.6 Object Diagram

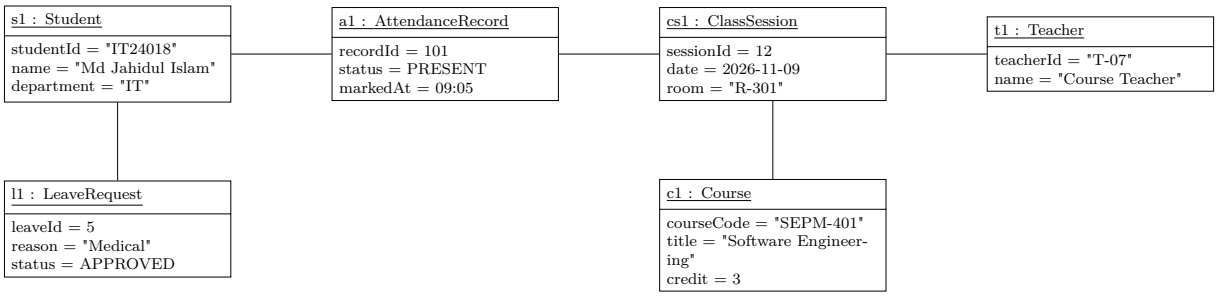


Figure 8: Object diagram: one snapshot of the running system (sample values).

7 Prototype

The prototype is a simple interface (JavaFX desktop or Javalin web page) built on five classes of the class diagram: `Student`, `Teacher`, `Course`, `ClassSession` and `AttendanceRecord`. `LeaveRequest` is added in Sprint 4.

Class	Role in prototype
Student	Logs in, marks attendance, sees own percentage
Teacher	Opens/closes a session, sees the attendance list
Course	Groups sessions and computes course attendance
ClassSession	Represents one class meeting, open or closed
AttendanceRecord	Stores one student's status for one session

Core class (excerpt).

```
public class AttendanceRecord {
    public enum Status { PRESENT, ABSENT, LATE, EXCUSED }

    private final Student student;
    private final ClassSession session;
    private Status status;

    public AttendanceRecord(Student student, ClassSession session, Status status
        ) {
        if (!session.isOpen()) throw new IllegalStateException("Session is
            closed");
        this.student = student;
        this.session = session;
        this.status = status;
    }

    public boolean isPresent() {
        return status == Status.PRESENT || status == Status.LATE;
    }
}
```

Interface screenshot.

Place the prototype screenshot at `screenshots/ui.png`

Figure 9: Prototype interface.

Repository structure.

```
smart-attendance-proposal/
  proposal.tex           (this document)
  screenshots/ui.png
  diagrams/              (UML source files)
  prototype/
    src/main/java/    (Student, Teacher, Course, ClassSession, AttendanceRecord,
         Main)
    pom.xml
  README.md
```

8 Testing and Quality Plan

- **Unit tests** (JUnit 5) for percentage calculation, session open/close rules and leave approval.
- **Integration tests** for the REST endpoints and database access.
- **User acceptance test** with at least five classmates acting as students and one as teacher.
- **Quality practices:** code review via pull requests, one feature per branch, sprint retrospectives.

9 Conclusion

The QR Code Based Attendance System reduces class time spent on roll call, removes manual percentage calculation, gives students early warnings and gives administrators reliable reports. The Scrum plan delivers a working increment every two weeks, and the prototype demonstrates the core classes defined in the UML design.

References

[1] I. Sommerville, *Software Engineering*, 10th ed., Pearson, 2016.

[2] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, McGraw-Hill.

[3] K. Schwaber and J. Sutherland, *The Scrum Guide*, 2020.

[4] OMG, *Unified Modeling Language Specification*, v2.5.1.